

# spark

June 7, 2026

## 1 Spark Exercise

Apache Spark is an excellent tool for data engineering projects due to its robust ability to process large-scale data efficiently through distributed computing. Spark's in-memory processing capabilities significantly enhance the speed of data operations, making it ideal for handling big data workloads. It supports various data sources and formats, offering versatility in data ingestion and transformation. Additionally, Spark's rich API supports multiple programming languages such as Python, Java, and Scala, catering to diverse developer preferences. Its ecosystem, which includes libraries for SQL, machine learning, and graph processing, provides a comprehensive suite for building complex data pipelines and analytics, making it a powerful and flexible choice for data engineering tasks.

Use Python, `pyspark` and `pandas` to explore Apache Spark RDD and DataFrame:

## 2 Spark RDD

Spark RDD (Resilient Distributed Dataset) is a fundamental data structure in Apache Spark that enables fault-tolerant, distributed processing of large datasets across multiple nodes in a cluster. Spark RDDs provide a higher-level abstraction for performing distributed data processing tasks, including both map (transformations) and reduce (aggregations) operations.

### 2.1 Import Necessary Libraries

```
[1]: # Import necessary libraries
import json
from pyspark import SparkContext
from pyspark.sql import SparkSession, Row
from pyspark.sql.functions import to_date, col, avg, count, max, min, explode,
    ↪ arrays_zip
import pandas as pd
import os

# Path to DATA
PATH_DATA = "data/raw/weather_raw_20260423_1641_start2013-01-01_end2025-12-31.
    ↪ json"
PATH_DATA_PROCESSED = "data/processed/
    ↪ weather_processed_20260423_1641_start2013-01-01_end2025-12-31.parquet"
```

```

# Verify paths exist
# Quick Overview
if os.path.exists(PATH_DATA):
    print("Data file exists")
    weather_df = pd.read_json(PATH_DATA)
    display(weather_df)

    print("\033[31mJSON must be sanitized!\033[0m")
else:
    print("\033[31mData file does NOT exists!\033[0m")

```

Data file exists

|                      | latitude  | longitude | generationtime_ms \ |
|----------------------|-----------|-----------|---------------------|
| time                 | 48.189804 | 16.377296 | 2354.54905          |
| temperature_2m       | 48.189804 | 16.377296 | 2354.54905          |
| relative_humidity_2m | 48.189804 | 16.377296 | 2354.54905          |
| wind_speed_10m       | 48.189804 | 16.377296 | 2354.54905          |
| wind_direction_10m   | 48.189804 | 16.377296 | 2354.54905          |
| precipitation        | 48.189804 | 16.377296 | 2354.54905          |
| shortwave_radiation  | 48.189804 | 16.377296 | 2354.54905          |
| surface_pressure     | 48.189804 | 16.377296 | 2354.54905          |

|                      | utc_offset_seconds | timezone      | timezone_abbreviation \ |
|----------------------|--------------------|---------------|-------------------------|
| time                 | 7200               | Europe/Vienna | GMT+2                   |
| temperature_2m       | 7200               | Europe/Vienna | GMT+2                   |
| relative_humidity_2m | 7200               | Europe/Vienna | GMT+2                   |
| wind_speed_10m       | 7200               | Europe/Vienna | GMT+2                   |
| wind_direction_10m   | 7200               | Europe/Vienna | GMT+2                   |
| precipitation        | 7200               | Europe/Vienna | GMT+2                   |
| shortwave_radiation  | 7200               | Europe/Vienna | GMT+2                   |
| surface_pressure     | 7200               | Europe/Vienna | GMT+2                   |

|                      | elevation | hourly_units \   |
|----------------------|-----------|------------------|
| time                 | 179       | iso8601          |
| temperature_2m       | 179       | °C               |
| relative_humidity_2m | 179       | %                |
| wind_speed_10m       | 179       | km/h             |
| wind_direction_10m   | 179       | °                |
| precipitation        | 179       | mm               |
| shortwave_radiation  | 179       | W/m <sup>2</sup> |
| surface_pressure     | 179       | hPa              |

|                      | hourly                                            |
|----------------------|---------------------------------------------------|
| time                 | [2013-01-01T00:00, 2013-01-01T01:00, 2013-01-0... |
| temperature_2m       | [-1.3, -1.4, -0.9, -0.4, -0.1, -0.1, -0.300000... |
| relative_humidity_2m | [84, 83, 80, 76, 74, 74, 76, 78, 82, 84, 85, 8... |
| wind_speed_10m       | [5.4, 6.0, 5.5, 6.0, 6.9, 7.7, 6.6, 5.1, 5.3, ... |

```
wind_direction_10m    [172, 155, 148, 147, 137, 131, 135, 129, 118, ...
precipitation         [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ...
shortwave_radiation   [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ...
surface_pressure       [994.9, 994.2, 993.8, 993.2, 992.7, 992.0, 991...
```

JSON must be sanitized!

## 2.2 Spark Context and Session

Initialize Spark Context and Spark Session

```
[2]: spark = SparkSession.builder \
      .appName("EX03") \
      .master("local") \
      .getOrCreate()
sc = spark.sparkContext
```

## 2.3 Load Data into RDD

```
[3]: # Read the file as an RDD (line by line)
rdd_raw = sc.textFile(PATH_DATA)

# Join all lines into one string (because the file is a single multi-line JSON_
↳object)
json_str = "\n".join(rdd_raw.collect())

# Parse the entire JSON document at once
data = json.loads(json_str)

# Extract the hourly block
hourly = data["hourly"]

# Build a list of dictionaries, one per hour
records = []
num_hours = len(hourly["time"])

for i in range(num_hours):
    record = {
        "time": hourly["time"][i],
        "temperature_2m": hourly["temperature_2m"][i],
        "relative_humidity_2m": hourly["relative_humidity_2m"][i],
        "wind_speed_10m": hourly["wind_speed_10m"][i],
        "wind_direction_10m": hourly["wind_direction_10m"][i],
        "precipitation": hourly["precipitation"][i],
        "shortwave_radiation": hourly["shortwave_radiation"][i],
        "surface_pressure": hourly["surface_pressure"][i]
    }
    records.append(record)
```

```

# Convert the list of hourly records into an RDD
rdd_data = sc.parallelize(records)

# Save the number of records for comparison
num_hours_before_map_reduce = rdd_data.count()

# Test output
print("Total hourly records:", num_hours_before_map_reduce)
print("Sample record:", rdd_data.take(1))

```

Total hourly records: 113952  
Sample record: [{'time': '2013-01-01T00:00', 'temperature\_2m': -1.3, 'relative\_humidity\_2m': 84, 'wind\_speed\_10m': 5.4, 'wind\_direction\_10m': 172, 'precipitation': 0.0, 'shortwave\_radiation': 0.0, 'surface\_pressure': 994.9}]

## 2.4 Map Operation

Split data into individual parts and create key-value pairs

```

[4]: # Map each hourly record into a flat structure
# -> {"date": "YYYY-MM-DD", "temperature_2m": ..., "wind_speed_10m": ..., ...}

def flatten_record(rec):
    date = rec["time"][:10]
    flat = {} # <-- remove date here
    for k, v in rec.items():
        if k != "time" and isinstance(v, (int, float)):
            flat[k] = v
    return (date, flat)

rdd_flat = rdd_data.map(flatten_record)

# Convert into (date, (flat_record, 1)) for aggregation
rdd_daily_numeric = rdd_flat.map(
    lambda x: (x[0], (x[1], 1))
)

rdd_daily_numeric.takeSample(num = 2, withReplacement = False)

```

```

[4]: [('2020-06-13',
      ({'temperature_2m': 19.6,
        'relative_humidity_2m': 76,
        'wind_speed_10m': 4.0,
        'wind_direction_10m': 190,
        'precipitation': 0.0,
        'shortwave_radiation': 0.0,
        'surface_pressure': 987.9},

```

```

1)),
('2024-05-06',
({'temperature_2m': 19.7,
 'relative_humidity_2m': 64,
 'wind_speed_10m': 7.2,
 'wind_direction_10m': 177,
 'precipitation': 0.0,
 'shortwave_radiation': 50.0,
 'surface_pressure': 983.8},
1))]
```

## 2.5 Reduce Operation

Reduce your key-value pairs

```

[5]: # Reduce by date: sum all numeric fields and counts
def merge(a, b):
    da, ca = a
    db, cb = b
    merged = {}
    for k in da:
        merged[k] = da[k] + db[k]
    return (merged, ca + cb)

rdd_daily_sums = rdd_daily_numeric.reduceByKey(merge)

# Compute averages: divide each sum by count
def compute_avg(x):
    sums, count = x
    return {k: sums[k] / count for k in sums}

rdd_daily_avg = rdd_daily_sums.mapValues(compute_avg)
rdd_daily_avg.takeSample(num = 2, withReplacement = False)
```

```

[5]: [('2022-12-22',
 {'temperature_2m': -0.15416666666666667,
 'relative_humidity_2m': 93.29166666666667,
 'wind_speed_10m': 8.820833333333333,
 'wind_direction_10m': 176.29166666666666,
 'precipitation': 0.050000000000000001,
 'shortwave_radiation': 17.333333333333332,
 'surface_pressure': 993.05000000000001}),
 ('2025-09-08',
 {'temperature_2m': 18.079166666666667,
 'relative_humidity_2m': 73.66666666666667,
 'wind_speed_10m': 4.479166666666666,
 'wind_direction_10m': 327.20833333333333,
```

```
'precipitation': 0.0,
'shortwave_radiation': 225.29166666666666,
'surface_pressure': 995.3708333333335}]]
```

## 2.6 Collect Results

Because of lazy evaluation, the map-reduce operation is performed only now. Show what you calculated.

```
[6]: # Collect results to driver (safe because daily results are small)
results_rdd = rdd_daily_avg.collect()

print(f"Total dates processed: {len(results_rdd)} (before map/reduce:
↳{num_hours_before_map_reduce})")
print("\nFirst 2 results (daily averages):")

for i, (date, avg_dict) in enumerate(results_rdd[:2], 1):
    print(f"{i}. Date: {date}")
    for key, value in avg_dict.items():
        print(f"    {key}: {value:.2f}")
```

Total dates processed: 4748 (before map/reduce: 113952)

First 2 results (daily averages):

1. Date: 2013-01-01
  - temperature\_2m: 0.09
  - relative\_humidity\_2m: 83.17
  - wind\_speed\_10m: 6.20
  - wind\_direction\_10m: 125.54
  - precipitation: 0.00
  - shortwave\_radiation: 54.33
  - surface\_pressure: 992.64
2. Date: 2013-01-02
  - temperature\_2m: 1.05
  - relative\_humidity\_2m: 90.67
  - wind\_speed\_10m: 9.94
  - wind\_direction\_10m: 221.71
  - precipitation: 0.03
  - shortwave\_radiation: 39.17
  - surface\_pressure: 1000.37

## 2.7 Save Results

```
[7]: # Convert RDD into Rows (date + all averaged fields)
df_daily_avg = spark.createDataFrame(
    rdd_daily_avg.map(lambda x: Row(date=x[0], **x[1]))
)
```

```

# Convert date column to proper DateType
df_daily_avg = df_daily_avg.withColumn("date", to_date("date", "yyyy-MM-dd"))

# Save as Parquet
output_path = PATH_DATA_PROCESSED.replace(".parquet", "_rdd_daily_avg.parquet")
df_daily_avg.write.mode("overwrite").parquet(output_path)

print(f"Parquet file saved to: {output_path}")

```

Parquet file saved to: data/processed/weather\_processed\_20260423\_1641\_start2013-01-01\_end2025-12-31\_rdd\_daily\_avg.parquet

### 3 Spark DataFrame

Spark DataFrame is a distributed collection of data organized into named columns, designed for efficient data manipulation and analysis in Apache Spark. It is used for various data processing tasks such as data ingestion, transformation, querying, and analysis in Apache Spark, providing a high-level abstraction that simplifies working with structured data.

#### 3.1 Load Data into DataFrame

```

[8]: # Load multiline JSON into a Spark DataFrame
df_raw = spark.read.option("multiline", "true").json(PATH_DATA)
print("JSON loaded successfully!")

# Flatten the hourly arrays into one row per hour
#   arrays_zip() combines all hourly arrays element-wise
#   explode() turns each element into its own row
df_hourly = df_raw.select(
    explode(
        arrays_zip(
            "hourly.time",
            "hourly.temperature_2m",
            "hourly.relative_humidity_2m",
            "hourly.wind_speed_10m",
            "hourly.wind_direction_10m",
            "hourly.precipitation",
            "hourly.shortwave_radiation",
            "hourly.surface_pressure"
        )
    ).alias("h")
)

# Select flattened hourly columns
df_hourly = df_hourly.select(
    col("h.time").alias("time"),
    col("h.temperature_2m"),

```

```

    col("h.relative_humidity_2m"),
    col("h.wind_speed_10m"),
    col("h.wind_direction_10m"),
    col("h.precipitation"),
    col("h.shortwave_radiation"),
    col("h.surface_pressure")
)

num_hours_before_agg = df_hourly.count()
print("DataFrame loaded successfully!")
print(f"Number of rows: {num_hours_before_agg}")

```

JSON loaded successfully!  
 DataFrame loaded successfully!  
 Number of rows: 113952

### 3.2 View DataFrame Schema

```

[9]: # Display DataFrame schema
print("DataFrame Schema:")
df_hourly.printSchema()

```

```

DataFrame Schema:
root
|-- time: string (nullable = true)
|-- temperature_2m: double (nullable = true)
|-- relative_humidity_2m: long (nullable = true)
|-- wind_speed_10m: double (nullable = true)
|-- wind_direction_10m: long (nullable = true)
|-- precipitation: double (nullable = true)
|-- shortwave_radiation: double (nullable = true)
|-- surface_pressure: double (nullable = true)

```

### 3.3 View DataFrame Data

```

[10]: # Convert time column to proper DateType
df_hourly = df_hourly.withColumn("date", to_date(col("time").substr(1, 10),
↪ "yyyy-MM-dd"))

df_hourly.select(
    'time',
    'temperature_2m',
    'relative_humidity_2m'
).describe().show()

# Display first rows

```

```
print("First 5 rows of the DataFrame:")
df_hourly.select("time", "date", "temperature_2m", "relative_humidity_2m").
↳ show(5, truncate=False)
```

```
+-----+-----+-----+-----+
|summary|      time| temperature_2m|relative_humidity_2m|
+-----+-----+-----+-----+
| count|    113952|          113952|          113952|
|  mean|      NULL|11.657856816905412|  71.85672914911542|
| stddev|      NULL| 8.682859934177936| 17.194267572673553|
|   min|2013-01-01T00:00|          -19.2|          13|
|   max|2025-12-31T23:00|          38.1|          100|
+-----+-----+-----+-----+
```

First 5 rows of the DataFrame:

```
+-----+-----+-----+-----+
|time      |date      |temperature_2m|relative_humidity_2m|
+-----+-----+-----+-----+
|2013-01-01T00:00|2013-01-01|-1.3          |84                  |
|2013-01-01T01:00|2013-01-01|-1.4          |83                  |
|2013-01-01T02:00|2013-01-01|-0.9          |80                  |
|2013-01-01T03:00|2013-01-01|-0.4          |76                  |
|2013-01-01T04:00|2013-01-01|-0.1          |74                  |
+-----+-----+-----+-----+
```

only showing top 5 rows

### 3.4 Filter Data

Performe a filter operation on a column

```
[11]: # Filtler Data
date_start = "2014-01-01"
date_end = "2024-12-31"

# DataFrame
print("""-----
DataFrame
-----""")

# Filter for date range 2013-01-01 to 2024-12-31
df_filtered = df_hourly.filter(
    (col("date") >= date_start) &
    (col("date") <= date_end)
)
```

```

print(f"Records in date range: {df_filtered.count()} (before filtering:
↳{num_hours_before_agg})")
print("\nFirst 5 filtered records:")
df_filtered.select("time", "date", "temperature_2m", "relative_humidity_2m").
↳show(5, truncate=False)

# SQL
print("""-----
SQL
-----""")

df_hourly.createOrReplaceTempView("tblWeather")
df_filtered_sql = spark.sql(f"SELECT * FROM tblWeather WHERE date BETWEEN
↳'{date_start}' AND '{date_end}'")

print(f"Records in date range: {df_filtered_sql.count()} (before filtering:
↳{num_hours_before_agg})")
print("\nFirst 5 filtered records:")
df_filtered_sql.select("time", "date", "temperature_2m",
↳"relative_humidity_2m").show(5, truncate=False)

```

-----  
DataFrame  
-----

Records in date range: 96432 (before filtering: 113952)

First 5 filtered records:

| time             | date       | temperature_2m | relative_humidity_2m |
|------------------|------------|----------------|----------------------|
| 2014-01-01T00:00 | 2014-01-01 | 4.2            | 95                   |
| 2014-01-01T01:00 | 2014-01-01 | 4.1            | 97                   |
| 2014-01-01T02:00 | 2014-01-01 | 4.1            | 97                   |
| 2014-01-01T03:00 | 2014-01-01 | 4.1            | 97                   |
| 2014-01-01T04:00 | 2014-01-01 | 4.0            | 98                   |

only showing top 5 rows

-----  
SQL  
-----

Records in date range: 96432 (before filtering: 113952)

First 5 filtered records:

| time             | date       | temperature_2m | relative_humidity_2m |
|------------------|------------|----------------|----------------------|
| 2014-01-01T00:00 | 2014-01-01 | 4.2            | 95                   |

|                                 |    |  |
|---------------------------------|----|--|
| 2014-01-01T01:00 2014-01-01 4.1 | 97 |  |
| 2014-01-01T02:00 2014-01-01 4.1 | 97 |  |
| 2014-01-01T03:00 2014-01-01 4.1 | 97 |  |
| 2014-01-01T04:00 2014-01-01 4.0 | 98 |  |

+-----+-----+-----+-----+-----+  
only showing top 5 rows

### 3.5 Group By and Aggregate

Performe a group by and aggregat operation

```
[12]: # Group by day and calculate aggregate statistics
# DataFrame
print("""-----
DataFrame
-----""")

df_agg = df_hourly.groupBy("date").agg(
    avg("temperature_2m").alias("avg_temperature_2m"),
    avg("relative_humidity_2m").alias("avg_relative_humidity_2m"),
    avg("wind_speed_10m").alias("avg_wind_speed_10m"),
    avg("wind_direction_10m").alias("avg_wind_direction_10m"),
    avg("precipitation").alias("avg_precipitation"),
    avg("shortwave_radiation").alias("avg_shortwave_radiation"),
    max("surface_pressure").alias("max_surface_pressure"),
    count("temperature_2m").alias("record_count")
).orderBy("date")

print(f"Total dates processed: {df_agg.count()} (before aggregation: {
    ↪{num_hours_before_agg})")
print("\nFirst 5 results (daily averages):")
df_agg.select("date", "avg_temperature_2m", "avg_relative_humidity_2m",
    ↪"record_count").show(5, truncate=False)

# SQL
print("""-----
SQL
-----""")

df_hourly.createOrReplaceTempView("tblWeather")
df_agg_sql = spark.sql("""
SELECT
date,
AVG(temperature_2m) AS avg_temperature_2m,
AVG(relative_humidity_2m) AS avg_relative_humidity_2m,
AVG(wind_speed_10m) AS avg_wind_speed_10m,
AVG(wind_direction_10m) AS avg_wind_direction_10m,
```

```

AVG(precipitation) AS avg_precipitation,
AVG(shortwave_radiation) AS avg_shortwave_radiation,
MAX(surface_pressure) AS max_surface_pressure,
COUNT(temperature_2m) AS record_count
FROM tblWeather
GROUP BY date
ORDER BY date"")

print(f"Total dates processed: {df_agg_sql.count()} (before aggregation:␣
↳{num_hours_before_agg})")
print("\nFirst 5 results (daily averages):")
df_agg_sql.select("date", "avg_temperature_2m", "avg_relative_humidity_2m",␣
↳"record_count").show(5, truncate=False)

```

-----  
DataFrame  
-----

Total dates processed: 4748 (before aggregation: 113952)

First 5 results (daily averages):

| date       | avg_temperature_2m  | avg_relative_humidity_2m | record_count |
|------------|---------------------|--------------------------|--------------|
| 2013-01-01 | 0.09166666666666663 | 83.16666666666667        | 24           |
| 2013-01-02 | 1.05                | 90.66666666666667        | 24           |
| 2013-01-03 | 3.0208333333333335  | 75.83333333333333        | 24           |
| 2013-01-04 | 7.741666666666666   | 80.25                    | 24           |
| 2013-01-05 | 7.787500000000001   | 85.08333333333333        | 24           |

only showing top 5 rows

-----  
SQL  
-----

Total dates processed: 4748 (before aggregation: 113952)

First 5 results (daily averages):

| date       | avg_temperature_2m  | avg_relative_humidity_2m | record_count |
|------------|---------------------|--------------------------|--------------|
| 2013-01-01 | 0.09166666666666663 | 83.16666666666667        | 24           |
| 2013-01-02 | 1.05                | 90.66666666666667        | 24           |
| 2013-01-03 | 3.0208333333333335  | 75.83333333333333        | 24           |
| 2013-01-04 | 7.741666666666666   | 80.25                    | 24           |
| 2013-01-05 | 7.787500000000001   | 85.08333333333333        | 24           |

only showing top 5 rows

### 3.6 Save DataFrame to Parquet

```
[13]: # Save DataFrame to Parquet format
agg_output_path = PATH_DATA_PROCESSED.replace('.parquet', '_dataframe_daily_avg.
↳parquet')
df_agg.write.mode("overwrite").parquet(agg_output_path)
print(f"DataFrame aggregation results saved to: {agg_output_path}")

# Also save the filtered data
filtered_output_path = agg_output_path.replace('daily_avg', 'filtered')
df_filtered.write.mode("overwrite").parquet(filtered_output_path)
print(f"Filtered data saved to: {filtered_output_path}")
```

DataFrame aggregation results saved to: data/processed/weather\_processed\_20260423\_1641\_start2013-01-01\_end2025-12-31\_dataframe\_daily\_avg.parquet  
Filtered data saved to: data/processed/weather\_processed\_20260423\_1641\_start2013-01-01\_end2025-12-31\_dataframe\_filtered.parquet